گزارش پروژه دوم: پلتفرم بانکداری توزیع‌شده

سینا اخوی

آبان ۱۴۰۵

---

۱ مقدمه و معماری کلان سیستم

در دنیای مدرن مهندسی نرم‌افزار، با افزایش حجم تراکنش‌ها و نیاز به دسترسی‌پذیری بالا (High Availability)، پایگاه‌های داده یکپارچه (Monolithic) دیگر پاسخگوی نیازهای سیستم‌های مالی مقیاس‌پذیر نیستند. در این پروژه، یک پلتفرم بانکداری توزیع‌شده طراحی و پیاده‌سازی شده است که قادر است بار پردازشی را میان چندین سرور و پایگاه داده مستقل تقسیم کند.

بر اساس قضیه CAP، در سیستم‌های توزیع‌شده هنگام بروز قطعی شبکه (Partition Tolerance)، باید بین سازگاری (Consistency) و دسترسی‌پذیری (Availability) یک مورد را انتخاب کرد. از آنجایی که این سیستم یک پلتفرم مالی است، معماری ما روی Consistency (یکپارچگی و سازگاری داده‌ها) متمرکز شده است تا از خطاهایی نظیر تولید پول بدون پشتوانه یا دو بار خرج کردن (Double Spending) جلوگیری شود.

این پلتفرم شامل یک دروزاه متمرکز (Gateway)، کلاسترهای پردازشی مستقل (Replicas) و پایگاه‌های داده مجزا است که به صورت کانتینرایز شده (Containerized) روی بستر Docker پیاده‌سازی شده‌اند. در بخش‌های بعدی، به تشریح دقیق الگوهای توزیع داده، مدیریت همزمانی و پروتکل‌های تحمل خطا خواهیم پرداخت.

۲ شاردینگ و مسیریابی (Sharding & Routing)

به منظور توزیع بار ذخیره‌سازی و پردازش روی دیتابیس‌ها، از تکنیک شاردینگ استفاده شده است. شاردینگ در واقع تقسیم‌بندی افقی (Horizontal Partitioning) داده‌هاست. در این سیستم، داده‌های حساب‌های بانکی بر اساس یک منطق قطعی در دو شارد کاملاً مستقل توزیع شده‌اند. برای این منظور، از یک رویکرد مبتنی بر محدوده (Range-based Sharding) استفاده شده است که نسبت به روش‌های مبتنی بر هش (Hash-based) مزایایی نظیر سادگی در پیاده‌سازی و قابلیت پیش‌بینی مسیریابی را به همراه دارد:

- **Shard A:** مالکیت حساب‌های A0001 تا A1000 را بر عهده دارد و داده‌ها در Database A ذخیره می‌شوند.

- **Shard B:** مالکیت حساب‌های A1001 تا A2000 را بر عهده دارد و داده‌ها در Database B ذخیره می‌شوند.

در این معماری، کلاینت‌ها کاملاً از ساختار زیرین و تعداد شاردها بی‌اطلاع هستند (Transparency). دروازه مسیریاب (Gateway) به عنوان یک پروکسی معکوس عمل کرده و با دریافت شناسه حساب از کاربر، با استفاده از نقشه شاردها (Shard Map) که در حافظه خود دارد، تصمیم‌گیری می‌کند که ترافیک باید به سمت کدام پایگاه داده و کلاستر هدایت شود.

۳ توزیع بارکاری و تحمل خطا (Load Balancing & Failover)

برای جلوگیری از ایجاد نقطه شکست منفرد (Single Point of Failure) در سطح اپلیکیشن، هر شارد دارای حداقل دو گره پردازشی بدون حالت (Stateless) است. به عنوان مثال، سرورهای A1 و A2 هر دو کدهای یکسانی را اجرا کرده و به دیتابیس مشترک A متصل هستند.

دروازه برای توزیع ترافیک و متعادل‌سازی بار بین این گره‌های سالم، از الگوریتم **Round-Robin** استفاده می‌کند. این پیاده‌سازی از طریق شیء itertools.cycle در پایتون انجام شده است که درخواست‌ها را به صورت متناوب بین رپلیکاهای موجود توزیع می‌کند. از آنجایی که سرورها بدون حالت هستند، نیازی به مکانیزم پیچیده Sticky Sessions وجود ندارد.

علاوه بر توزیع بار، Gateway مکانیزم تحمل خطا را نیز پیاده‌سازی کرده است. چنانچه درخواستی به یکی از گره‌ها (مثلاً A1) ارسال شود و به دلیل خرابی سرور یا قطعی شبکه خطای Timeout یا Connection Refused دریافت گردد، هماهنگ‌کننده به صورت خودکار ترافیک را به گره بعدی در چرخه (A2) رله می‌کند، به طوری که کلاینت هرگز متوجه قطعی سرور پشت صحنه نخواهد شد.

۴ کنترل همزمانی و جلوگیری از بن‌بست (Concurrency Control)

یکی از بزرگترین چالش‌ها در سیستم‌های توزیع‌شده با چند رپلیکا، پدیده شرایط رقابتی (Race Condition) و به‌روزرسانی‌های از دست رفته (Lost Update) است. اگر دو کاربر همزمان تلاش کنند از یک حساب پول برداشت کنند و هر دو گره A1 و A2 به طور همزمان موجودی اولیه را بخوانند، موجودی نهایی دچار مغایرت خواهد شد.

برای حل این مشکل، از مکانیزم قفل‌گذاری بدبینانه در سطح سطر (Pessimistic Row-level Locking) در PostgreSQL استفاده شده است. با استفاده از دستور SELECT ... FOR UPDATE، هنگامی که یک تراکنش برداشت یا واریز آغاز می‌شود، سطر مربوط به آن حساب در دیتابیس قفل (Lock) می‌شود. در نتیجه، رپلیکاهای دیگر تا زمان اجرای عملیات COMMIT یا ROLLBACK توسط گره اول، در صف انتظار مسدود (Block) می‌مانند.

همچنین، برای جلوگیری از پدیده بن‌بست (Deadlock) در تراکنش‌های محلی که دو حساب همزمان با یکدیگر درگیر هستند، سیستم پیش از قفل کردن سطرها، شناسه حساب‌ها را مرتب‌سازی (Sort) می‌کند تا قفل‌گذاری همیشه با یک ترتیب قطعی و الفبایی انجام شود و از قفل‌های حلقوی جلوگیری گردد.

۵ تراکنش‌های توزیع‌شده و پروتکل 2PC

پیچیده‌ترین بخش یک پلتفرم بانکی شارد شده، انتقال وجه بین دو شارد مختلف است (مانند ارسال پول از A0500 در دیتابیس اول به A1500 در دیتابیس دوم). از آنجایی که پایگاه‌های داده کاملاً مستقل هستند، نمی‌توان از تراکنش‌های استاندارد SQL استفاده کرد. به همین دلیل از پروتکل تعهد دو فازی (Two-Phase Commit) استفاده شده است که در آن Gateway به عنوان هماهنگ‌کننده (Coordinator) عمل می‌کند:

- **فاز اول (Prepare):** هماهنگ‌کننده دستور آماده‌سازی را با یک شناسه تراکنش یکتا (txn_id) به هر دو شارد ارسال می‌کند. شارد ارسال‌کننده (مبدا) موجودی را چک کرده و در صورت کافی بودن، مبلغ را از حساب کسر می‌کند اما آن را در حالت PREPARE TRANSACTION نگه می‌دارد (ثبت در فضای pg_prepared_xacts). شارد دریافت‌کننده نیز عملیات مشابهی را انجام می‌دهد. هر دو گره رأی YES خود را به Gateway برمی‌گردانند.
- **فاز دوم (Commit):** اگر هماهنگ‌کننده از تمام طرفین رأی مثبت دریافت کند، وضعیت تراکنش را در یک پایگاه داده داخلی از نوع SQLite به عنوان Write-Ahead Log با وضعیت COMMIT_DECIDED ذخیره می‌کند. این ثبت روی دیسک تضمین می‌کند که حتی در صورت قطع برق، تصمیم نهایی حفظ شود. سپس فرمان COMMIT PREPARED را برای شاردها ارسال می‌کند تا پول به طور قطعی جابه‌جا شود.

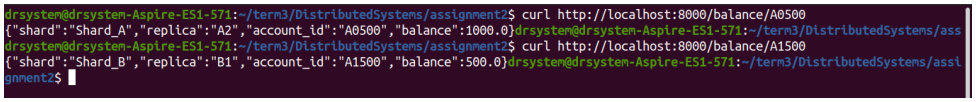
لازم به ذکر است که تمامی پایانه‌های (Endpoints) بانکی به صورت ادامپوتنت (Idempotent) طراحی شده‌اند. به این معنا که اگر هماهنگ‌کننده یک دستور COMMIT را به دلیل قطعی شبکه چند بار ارسال کند، سرور بانکی متوجه تکراری بودن شناسه تراکنش شده و از اعمال مجدد آن خودداری می‌کند.

۶ نتایج آزمایش‌های عملیاتی و ارزیابی سیستم

به منظور اثبات صحت عملکرد سیستم، شش آزمایش اجباری در محیط داکر با موفقیت اجرا گردید که در ادامه مستندات و تحلیل هر یک آورده شده است.

۱.۶ آزمایش ۱: بررسی توزیع و مسیریابی شاردها (Shard Distribution)

هدف از این آزمایش، اطمینان از عملکرد صحیح Shard Map در دروازه است. همانطور که در تصویر زیر مشاهده می‌شود، با درخواست موجودی حساب A0500، سیستم ترافیک را به Shard A هدایت می‌کند و حساب A1500 به طور دقیق توسط Shard B پاسخ داده می‌شود.

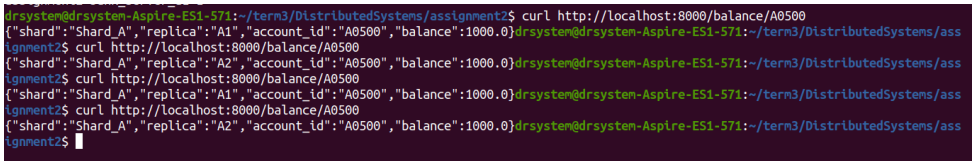


شکل ۱: هدایت صحیح درخواست‌های موجودی بر اساس منطق شاردینگ

۲.۶ آزمایش ۲: توزیع بارکاری بین رپلیکاها (Replica Load Distribution)

در این آزمایش چندین درخواست متوالی برای یک حساب مشخص ارسال شده است.

تحلیل نتیجه: همانطور که در خروجی‌های JSON در تصویر زیر قابل مشاهده است، مقدار فیلد replica به صورت منظم بین A1 و A2 تغییر می‌کند. این امر نشان‌دهنده عملکرد بی‌نقص الگوریتم Round-Robin در تقسیم مساوی فشار پردازشی میان گره‌های کلاستر است.

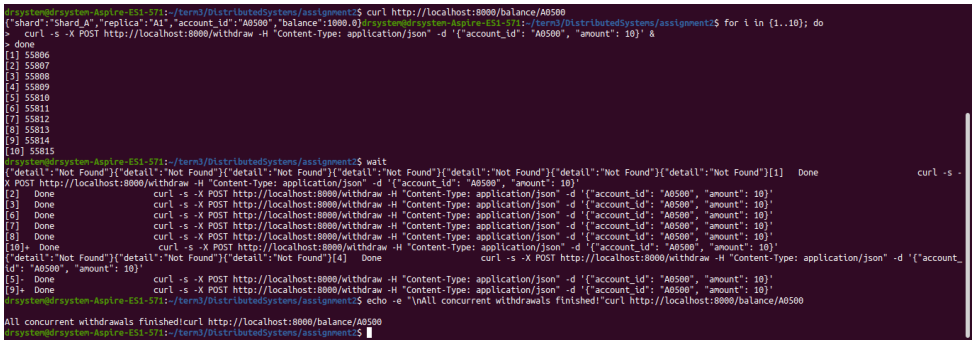


شکل ۲: تناوب پاسخ‌گویی بین رپلیکاهای A1 و A2 تحت بار کاری

۳.۶ آزمایش ۳: عملیات همزمان و شرایط رقابتی (Concurrent Replica Operations)

برای این آزمایش، یک اسکریپت Bash نوشته شد که دقیقاً در یک لحظه، ۱۰ درخواست برداشت مستقل (هر کدام به مبلغ ۱۰ واحد) را برای حساب A0500 ارسال می‌کند.

تحلیل نتیجه: موجودی اولیه ۹۰۰ واحد بوده است. در صورت عدم وجود قفل‌گذاری صحیح (FOR UPDATE)، رپلیکاها موجودی را بازنویسی کرده و عدد نهایی اشتباه می‌شد. اما تصویر نشان می‌دهد موجودی دقیقاً به ۸۰۰ واحد کاهش یافته است که اثبات می‌کند دیتابیس توانسته تراکنش‌های همزمان را به صورت کاملاً امن سری‌سازی (Serialize) کند.

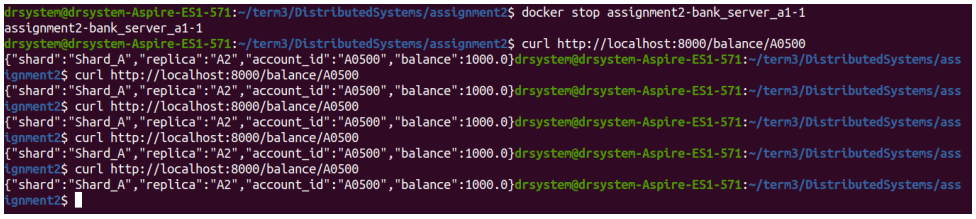


شکل ۳: موفقیت تراکنش‌های همزمان بدون بروز مغایرت در موجودی نهایی

۴.۶ آزمایش ۴: تحمل خطا و وقفه در رپلیکا (Replica Interruption)

در این آزمایش، با استفاده از دستور docker stop، یکی از گره‌های در حال کار (سرور A1) متوقف گردید.

تحلیل نتیجه: پس از قطع گره، کلاینت هیچ‌گونه توقفی در سرویس‌دهی احساس نکرد. Gateway با دریافت خطای اتصال از سمت گره مرده، سریعاً استراتژی Failover را فعال کرده و تمامی درخواست‌ها را به گره باقیمانده یعنی A2 هدایت کرد. تصویر زیر گویای این رفتار خودکار است.



شکل ۴: تغییر مسیر ترافیک به گره سالم پس از شبیه‌سازی قطعی سخت‌افزاری

۵.۶ آزمایش ۵: انتقال‌های محلی و بین‌شاردی (Local & Cross-shard Transfers)

این آزمایش نشان می‌دهد که سیستم به درستی میان انتقال وجه داخل یک دیتابیس واحد و انتقال وجه پیچیده بین دو دیتابیس مجزا تفکیک قائل شده است.

تحلیل نتیجه: تراکنش محلی (بین دو حساب در یک شارد) با استفاده از مکانیسم‌های داخلی دیتابیس و تراکنش‌های بین شاردی با تکیه بر Coordinator انجام شد. هر دو درخواست با کد وضعیت HTTP 200 و پیام موفقیت‌آمیز به پایان رسیدند.

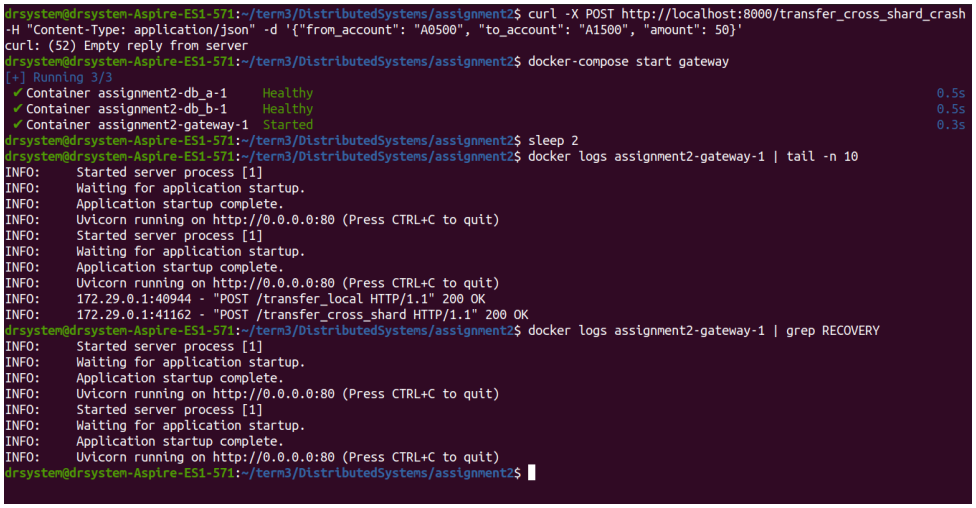


شکل ۵: اجرای موفقیت‌آمیز انتقال وجه در دو سناریوی Local و Cross-Shard

۶.۶ آزمایش ۶: وقفه در تراکنش توزیع‌شده (Interrupted 2PC Transaction)

مهم‌ترین سناریوی بحرانی در پروتکل 2PC زمانی رخ می‌دهد که هماهنگ‌کننده پس از ذخیره تصمیم نهایی روی دیسک، قطع شود (Crash). برای شبیه‌سازی این حالت، یک Endpoint مخصوص در کد تعبیه شد که بلافاصله پس از لاگ زدن COMMIT_DECIDED، به صورت تعمدی سرویس Gateway را مختل (os.exit) می‌کند.

تحلیل نتیجه: همانطور که در تصویر زیر مشخص است، کلاینت در ابتدا خطای Empty Reply دریافت کرد. اما پس از راه‌اندازی مجدد (Restart) کانتینر Gateway، سیستم به طور خودکار دیتابیس SQLite خود را اسکن کرده، تراکنش ناتمام را شناسایی نموده و با ارسال مجدد فاز دوم به گره‌ها، پروسه را به وضعیت COMPLETED رساند تا از بلوکه شدن دائمی پول‌ها جلوگیری شود.



شکل ۶: بازیابی موفقیت‌آمیز تراکنش 2PC و خروج از وضعیت بحرانی پس از Restart

۷ دسترسی به کدهای منبع

تمامی کدهای مربوط به این پروژه (شامل کدهای Gateway، رپلیکاها، تنظیمات Docker Compose و اسکریپت‌های تست) به صورت عمومی در مخزن گیت‌هاب نویسنده در دسترس قرار گرفته است:

https://github.com/Sina-Akhavi/distributed-systems-course/tree/main

۸ نتیجه‌گیری

پلتفرم پیاده‌سازی شده در این پروژه نشان داد که استفاده از معماری میکروسرویس در کنار پروتکل‌های سنتی اما قدرتمندی نظیر Two-Phase Commit می‌تواند پایداری (Consistency) مطلق را برای سیستم‌های مالی به ارمغان بیاورد. در این سامانه، مکانیزم‌های Failover توانستند دسترسی‌پذیری سیستم را در صورت بروز خرابی‌های سخت‌افزاری گره‌ها تضمین کنند، و پایگاه داده درونی Gateway مقاومت سیستم در برابر خطای هماهنگ‌کننده را اثبات کرد.

توسعه‌های آتی این پلتفرم می‌تواند شامل پیاده‌سازی الگوریتم اجماع Raft برای توزیع خودِ هماهنگ‌کننده (جلوگیری از Single Point of Failure در سطح Gateway) و پیاده‌سازی قابلیت Live Re-sharding برای انتقال داینامیک حساب‌ها بین شاردها بدون نیاز به قطعی سیستم باشد.